

Hybrid Volatility Surface Reconstruction

Integrating the SABR Model with TabPFN and Sobolev-Regularized Stacking

Joël Khayat Benjamin Benisti

Applied Quantitative Finance Research Project

June 3, 2026

Abstract

The SABR stochastic-volatility model is the market standard for quoting and interpolating implied-volatility smiles, but its closed-form Hagan expansion degrades in extreme regimes, and purely data-driven surrogates fail to deliver the smooth risk sensitivities (the *Greeks*) required for hedging. We study the reconstruction of an arbitrage-consistent, smooth SABR volatility surface that is simultaneously accurate in *value* and *geometry*. Our pipeline couples three components: (i) a fully differentiable JAX SABR engine that supplies exact analytical gradients as ground truth; (ii) TABPFN, a transformer-based prior-data-fitted network used as an in-context pricing oracle; and (iii) a deep multi-layer perceptron (MLP) “stacker” trained with a *Sobolev loss* that penalizes value and derivative errors jointly. To concentrate supervision where the surface curvature is highest, we introduce a *Hybrid Mesh* sampling scheme that blends global Sobol coverage with a Gaussian At-The-Money (ATM) refinement, and we select the network topology by automated search (Ray Tune with the Optuna sampler and the ASHA scheduler). Empirically, the baseline TABPFN attains near-perfect value accuracy ($\text{MAE} \approx 2.85 \times 10^{-5}$) yet a chaotic empirical skew; the Sobolev-regularized stacker removes this gradient noise and produces a predicted surface that overlaps the analytical lognormal SABR truth across the entire forward-strike domain, without ad-hoc post-processing. We discuss a diagnosed normal-vs-lognormal evaluation pitfall, the role of out-of-distribution control, and the limitations and reproducibility of the approach.

Keywords: SABR; implied volatility surface; TabPFN; in-context learning; Sobolev training; automatic differentiation; quasi-Monte Carlo; hyperparameter optimization; model stacking; quantitative finance.

1 Introduction

The implied-volatility surface is the central object of modern derivatives pricing and risk management. Practitioners rarely observe a dense, liquid surface; instead they must interpolate and extrapolate from sparse quotes while preserving no-arbitrage structure and stable sensitivities. The SABR model of Hagan et al. [1] has become the de facto market standard for this task because its four parameters $(\alpha, \beta, \rho, \nu)$ admit a transparent financial reading and because Hagan’s singular-perturbation expansion yields a closed-form implied volatility that is extremely fast to evaluate. Yet the same expansion is only first order: it can generate negative densities or static arbitrage at long maturities, in low- or negative-rate environments, and for deep out-of-the-money (OTM) strikes [2].

A natural idea is to learn the SABR map with a neural surrogate. Deep models have been used both to accelerate calibration and to learn pricing functions directly [9, 10]. Most such models, however, require an expensive gradient-descent training phase for each new dataset and, more importantly for risk management, produce *noisy first derivatives*. A model can be nearly exact on the volatility *level* while its numerical skew $\partial V / \partial K$ oscillates violently. Because hedging is driven by the Greeks — e.g. the delta $\Delta = \partial V / \partial F$ and the vega $\mathcal{V} = \partial V / \partial \alpha$, together with second-order quantities such as Vanna and Volga — a jagged gradient makes dynamic hedging impossible. We call this the *Gradient Challenge*.

This work attacks the problem along two complementary axes: *what the model learns from* (a curvature-aware data distribution) and *what the model is penalized on* (a differential, Sobolev-type objective). We use TABPFN [4] — a tabular foundation model that performs Bayesian *in-context learning* in a single forward pass — as a fast pricing oracle, and a deep MLP as a geometric filter that smooths and corrects the oracle output. The filter is supervised not only on prices but on the *exact* analytical gradients delivered by a differentiable JAX pricing engine [5], in the spirit of Sobolev training [6].

Contributions. We make the following contributions:

1. A **differentiable SABR engine** in JAX with explicit ATM singularity handling, providing exact, batched Greeks as ground truth for training and evaluation (Section 4).
2. A **Hybrid Mesh** data-generation scheme that combines low-discrepancy Sobol coverage of the parameter hypercube with a Gaussian ATM refinement, resolving boundary data starvation and concentrating density at the singularity (Section 8, Algorithm 1).
3. A **Sobolev-regularized stacking** architecture in which TABPFN acts as an oracle feature and an MLP minimizes a joint value/derivative loss, with an optional homoscedastic-uncertainty weighting (Sections 7 and 10, Algorithm 2).
4. An **automated architecture search** (Ray Tune + Optuna + ASHA) over MLP and tabular-transformer topologies, restricted to twice-differentiable activations to protect gradient flow (Section 9).
5. An **empirical study** showing that the stacker resolves the Gradient Challenge, together with a diagnosis of a normal-vs-lognormal evaluation pitfall and an out-of-distribution analysis (Section 11).

2 Related Work

SABR and its corrections. Hagan et al. [1] introduced the SABR model and its asymptotic implied-volatility expansion. Subsequent work corrected arbitrage artifacts of the original formula, notably the small-strike behavior [2] and arbitrage-free PDE/probability-density treatments [3]. These remain analytic or semi-analytic; our contribution is orthogonal, learning a smooth surrogate *supervised by* such an engine.

Deep learning for volatility. Neural networks have been applied to fast calibration and pricing of (rough) stochastic-volatility models [9–11]. These works mostly target the *value* map; the stability of the learned *derivatives* is less frequently enforced. We make derivative fidelity a first-class training objective.

Derivative-aware training. Sobolev training [6] augments the loss with target derivatives and provably improves sample efficiency and generalization for smooth target functions. Physics-informed and differential approaches [12, 13] similarly supervise gradients. We instantiate this idea with exact, autodiff-computed SABR sensitivities rather than noisy estimates.

Tabular foundation models. TABPFN [4] performs approximate Bayesian inference over tabular functions via in-context learning, reaching strong accuracy on small datasets without per-task training. We repurpose it as a pricing oracle inside a stacking ensemble, delegating geometric regularity to a downstream network.

Quasi-Monte Carlo and AutoML. Low-discrepancy Sobol sequences [8] provide uniform coverage superior to pseudo-random sampling; we add a targeted Gaussian refinement. Architecture and hyperparameters are selected with Ray Tune [14], the Optuna sampler [15] and the ASHA scheduler [16]. Adaptive multi-task weighting follows the homoscedastic-uncertainty formulation of Kendall et al. [7].

3 Background: The SABR Model

3.1 Dynamics and parameters

SABR is a two-factor stochastic-volatility model for a forward price F_t and its instantaneous volatility α_t :

$$dF_t = \alpha_t F_t^\beta dW_t^1, \quad (1)$$

$$d\alpha_t = \nu \alpha_t dW_t^2, \quad (2)$$

$$dW_t^1 dW_t^2 = \rho dt.$$

The parameters carry direct financial meaning: α sets the ATM level; β is the backbone, interpolating between Bachelier/normal ($\beta = 0$) and Black/lognormal ($\beta = 1$) dynamics; ρ controls the skew (asymmetry of the smile); and ν (“volvol”) controls the convexity (curvature). The learning target is the map

$$\Phi : (K, F, T, \alpha, \beta, \rho, \nu) \longmapsto \sigma_{\text{impl}}, \quad (3)$$

which a model must approximate so as to fill an illiquid surface instantly.

3.2 Hagan’s asymptotic expansion

For short maturities T and small volvol ν (bounded perturbation parameter $\varepsilon = \nu^2 T$), Hagan et al. [1] derived the lognormal (Black) implied volatility

$$\sigma_B(K, F) \approx \frac{\alpha}{(FK)^{\frac{1-\beta}{2}} \left[1 + \frac{(1-\beta)^2}{24} \ln^2 \frac{F}{K} + \frac{(1-\beta)^4}{1920} \ln^4 \frac{F}{K} \right]} \cdot \frac{z}{x(z)} \cdot [1 + \Delta T], \quad (4)$$

where the modified moneyness z and the skew function $x(z)$ are

$$z = \frac{\nu}{\alpha} (FK)^{\frac{1-\beta}{2}} \ln \frac{F}{K}, \quad x(z) = \ln \left(\frac{\sqrt{1 - 2\rho z + z^2} + z - \rho}{1 - \rho} \right), \quad (5)$$

and the higher-order bracket is

$$\Delta = \frac{(1-\beta)^2}{24} \frac{\alpha^2}{(FK)^{1-\beta}} + \frac{1}{4} \frac{\rho \beta \nu \alpha}{(FK)^{\frac{1-\beta}{2}}} + \frac{2-3\rho^2}{24} \nu^2. \quad (6)$$

The expression is extremely fast to evaluate, which is precisely why SABR dominates the quoting of swaptions and rate derivatives. Being a first-order approximation, however, it is known to break down in extreme regimes, producing arbitrage or negative implied densities [2].

3.3 The ATM singularity

As $K \rightarrow F$ we have $\ln(F/K) \rightarrow 0$, hence $z \rightarrow 0$ and $x(z) \rightarrow 0$, so the ratio $z/x(z)$ is a 0/0 indeterminate form.

Proposition 1 (ATM limit). *For $K = F$, the factor $z/x(z) \rightarrow 1$ and (4) reduces to*

$$\sigma_B^{\text{ATM}} = \frac{\alpha}{F^{1-\beta}} \left[1 + \left(\frac{(1-\beta)^2}{24} \frac{\alpha^2}{F^{2-2\beta}} + \frac{1}{4} \frac{\rho \beta \nu \alpha}{F^{1-\beta}} + \frac{2-3\rho^2}{24} \nu^2 \right) T \right]. \quad (7)$$

A faithful implementation must branch on (7) near $K = F$; otherwise the gradient evaluates to NaN and back-propagation explodes. Our engine codes this limit explicitly, ensuring continuity of both σ_B and its derivatives across the entire strike space.

4 Differentiable Pricing Engine

We implement (4)–(7) in JAX [5] using only `jax.numpy` primitives, so the whole pricing map is differentiable. Two engineering points are essential. First, because both branches of a `where` are evaluated under JAX tracing, we apply a safe perturbation to K on the non-ATM branch to avoid 0/0 contaminating the gradient with NaN. Second, we vectorize over all input axes with `vmap` and compile with `jit`, so that the engine prices and differentiates an entire batch of scenarios in one fused kernel.

Definition 1 (Analytical sensitivities). For a scenario $\theta = (K, F, T, \alpha, \beta, \rho, \nu)$, the engine returns the value $V = \sigma_B(\theta)$ together with the gradient vector $\nabla V = (\partial V / \partial K, \partial V / \partial F, \partial V / \partial \alpha, \partial V / \partial \rho, \partial V / \partial \nu, \partial V / \partial \beta)$, obtained exactly by reverse-mode automatic differentiation rather than finite differences.

These exact gradients serve two roles: they are the supervision target of the Sobolev loss (Section 7) and the benchmark against which the baseline’s geometric failure is measured (Section 6.3).

5 Synthetic Data Generation and Sobol Coverage

Because no single market provides enough labelled points to cover the SABR parameter space, we generate a synthetic dataset from the engine of Section 4. To explore the space without the clusters and voids of pseudo-random sampling, inputs are drawn from a Sobol low-discrepancy sequence [8] and mapped to $[-1, 1]$ for network stability. Table 1 lists the sampling bounds.

Table 1: Sampling bounds for synthetic data generation. Strikes are anchored to the forward through a moneyness factor for financial relevance.

Quantity	Symbol	Range
Backbone	β	[0.25, 0.99]
Spot–vol correlation	ρ	[−0.50, 0.50]
Volatility of volatility	ν	[0.15, 0.45]
Forward rate	F	[1%, 5%]
ATM normal volatility	σ_{ATM}^N	[50, 300] bps
Moneyness factor	K/F	[75%, 150%]

Structural integrity. Figure 1 shows the pairplot of the generated data. The diagonal histograms are uniform plateaus for β, ρ, ν , confirming Sobol uniformity. The concentration of α near zero is *not* a sampling bias: it follows from the non-linear inversion of Hagan’s formula from the uniform ATM-volatility space (we sample the ATM normal volatility and solve for α). The bivariate “cone” between F and K encodes the moneyness coupling, and near-zero Pearson correlations between the remaining parameters confirm their independence, so the downstream model learns each risk factor’s sensitivity without multicollinearity bias.

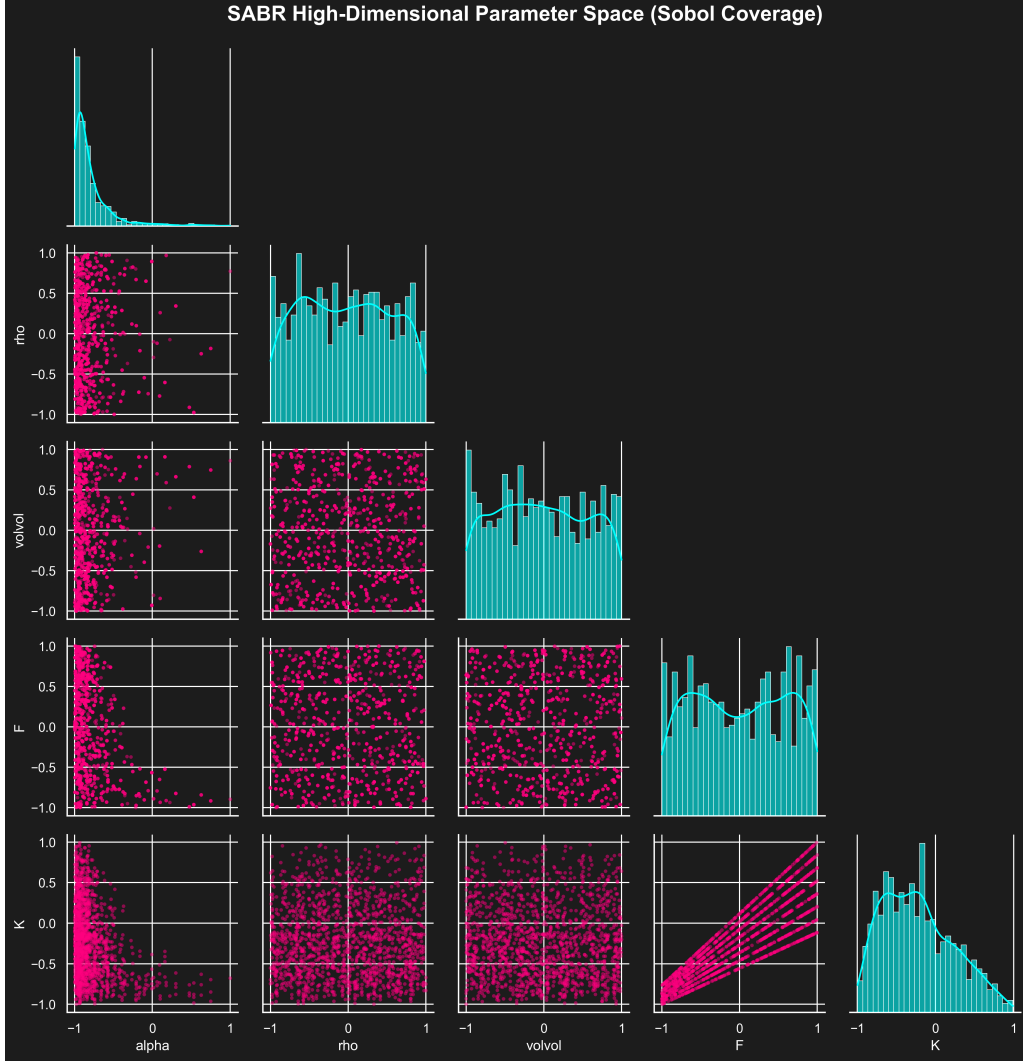


Figure 1: SABR parameter-space analysis. Diagonal histograms show marginal distributions; off-diagonal scatter plots reveal the structural dependencies induced by the moneyness coupling and the Hagan inversion.

6 Baseline: Standard TabPFN

6.1 Pre-processing

The pre-processing is tailored to the transformer’s attention mechanism: we add the log-moneyness coordinate $\ln(K/F)$, scale all inputs to $[-1, 1]$ with a `MinMaxScaler`, and scale the target volatility to $[0, 1]$. TABPFN is then used as an in-context regressor: a context set of labelled scenarios is supplied, and predictions on the query set are obtained in a single forward pass, without per-dataset training.

6.2 Value accuracy

On the value map, the baseline is excellent. As shown in Fig. 2, predicted and true implied volatilities align almost perfectly along the diagonal, with a Mean Absolute Error of $\text{MAE} \approx 2.85 \times 10^{-5}$. This confirms that the foundation model captures the SABR pricing function with high fidelity.

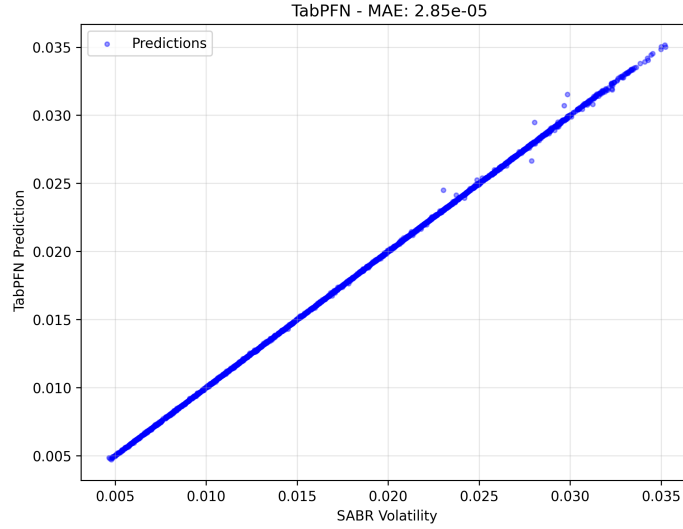


Figure 2: Baseline TABPFN predictions versus SABR ground truth for the implied volatility. Value accuracy is near-perfect.

6.3 The Gradient Challenge

Value accuracy does not imply geometric fidelity. Using the differentiable engine as the analytical reference and extracting the baseline’s empirical skew by finite differences, Fig. 3 shows that the TABPFN derivative oscillates wildly around the true trajectory (the two curves are z -score aligned to isolate shape). In a trading context these oscillations translate into unstable, costly hedging signals. The phenomenon — high accuracy in value, chaos in derivative — is the core obstacle this paper resolves.

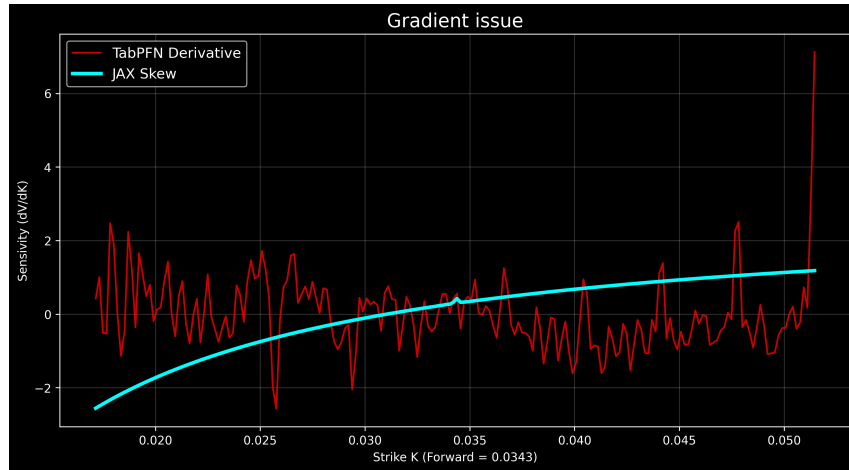


Figure 3: The Gradient Challenge. The baseline skew $\partial V / \partial K$ (noisy) against the exact JAX reference. High value accuracy coexists with a chaotic numerical derivative.

7 Sobolev-Regularized Learning

7.1 Objective

We move from point-wise regression to a Sobolev objective [6] that matches values *and* gradients. With predicted volatility $\hat{V}$, true value V , and predicted/true sensitivities $\nabla\hat{V}, \nabla V \in \mathbb{R}^d$, the loss is

$$\mathcal{L}_{\text{total}} = w_v \text{MAE}(\hat{V}, V) + w_d \frac{1}{N} \sum_{i=1}^N \text{MAE}(\nabla\hat{V}_i, \nabla V_i), \quad (8)$$

where $w_v, w_d > 0$ trade off price accuracy against surface smoothness. Crucially, $\nabla\hat{V}_i$ is obtained by differentiating the network through automatic differentiation, so the constraint acts directly on the model’s own Jacobian.

7.2 Adaptive weighting

Fixing (w_v, w_d) by hand is brittle. We optionally adopt the homoscedastic-uncertainty weighting of Kendall et al. [7]: with learnable log-variances s_v, s_d ,

$$\mathcal{L}_{\text{adapt}} = \frac{1}{2} e^{-s_v} \text{MAE}(\hat{V}, V) + \frac{1}{2} s_v + \frac{1}{2} e^{-s_d} \frac{1}{N} \sum_i \text{MAE}(\nabla\hat{V}_i, \nabla V_i) + \frac{1}{2} s_d, \quad (9)$$

so the optimizer learns the balance during training instead of relying on a static grid-searched ratio.

7.3 Granular monitoring

At each step we emit a per-Greek *loss breakdown*: rather than a single scalar we track MAE for every sensitivity $(\partial V / \partial K, \partial V / \partial \rho, \dots)$. This identifies which Greeks are hardest to learn — typically the skew near the ATM singularity — and provides the feedback used to tune w_d .

8 Hybrid Mesh Refinement

A uniform Sobol grid covers the hypercube well but cannot resolve SABR’s strongest non-linearity, the ATM region $K \approx F$ where curvature and gradients peak. We therefore use a **Hybrid Mesh** (Algorithm 1) that superposes a global Sobol layer with a hyper-dense Gaussian ATM layer drawn from $\ln(K/F) \sim \mathcal{N}(0, \sigma^2)$, $\sigma = 0.10$, so that roughly half of the data sits at $K/F \approx 1$.

Algorithm 1 Hybrid Mesh generation

Require: total points N , ATM ratio r , bounds $\mathcal{B}$, refinement width σ

- 1: $N_g \leftarrow \lfloor (1-r)N \rfloor$, $N_a \leftarrow N - N_g$
 - 2: $\mathbf{U} \leftarrow \text{SOBOL}(N_g, d=7)$ ▷ global low-discrepancy layer
 - 3: $X_g \leftarrow \text{RESCALE}(\mathbf{U}, \mathcal{B})$
 - 4: $\mathbf{V} \leftarrow \text{SOBOL}(N_a, d=6)$ ▷ all params except K
 - 5: $m \sim \mathcal{N}(0, \sigma^2)^{N_a}$, $K_a \leftarrow \text{CLIP}(F e^m, \mathcal{B}_K)$ ▷ ATM anchoring
 - 6: $X_a \leftarrow \text{ASSEMBLE}(\mathbf{V}, K_a)$
 - 7: $X \leftarrow \text{SHUFFLE}(X_g \cup X_a)$ ▷ balance PyTorch batches
 - 8: $(V, \nabla V) \leftarrow \text{JAXENGINE}(X)$ ▷ exact values & Greeks
 - 9: **return** $(X, V, \nabla V)$
-

As shown in Fig. 4, the refinement reallocates density toward the singularity while the Sobol layer keeps deep ITM/OTM boundaries populated, eliminating the data starvation that otherwise makes the network extrapolate blindly at the edges.

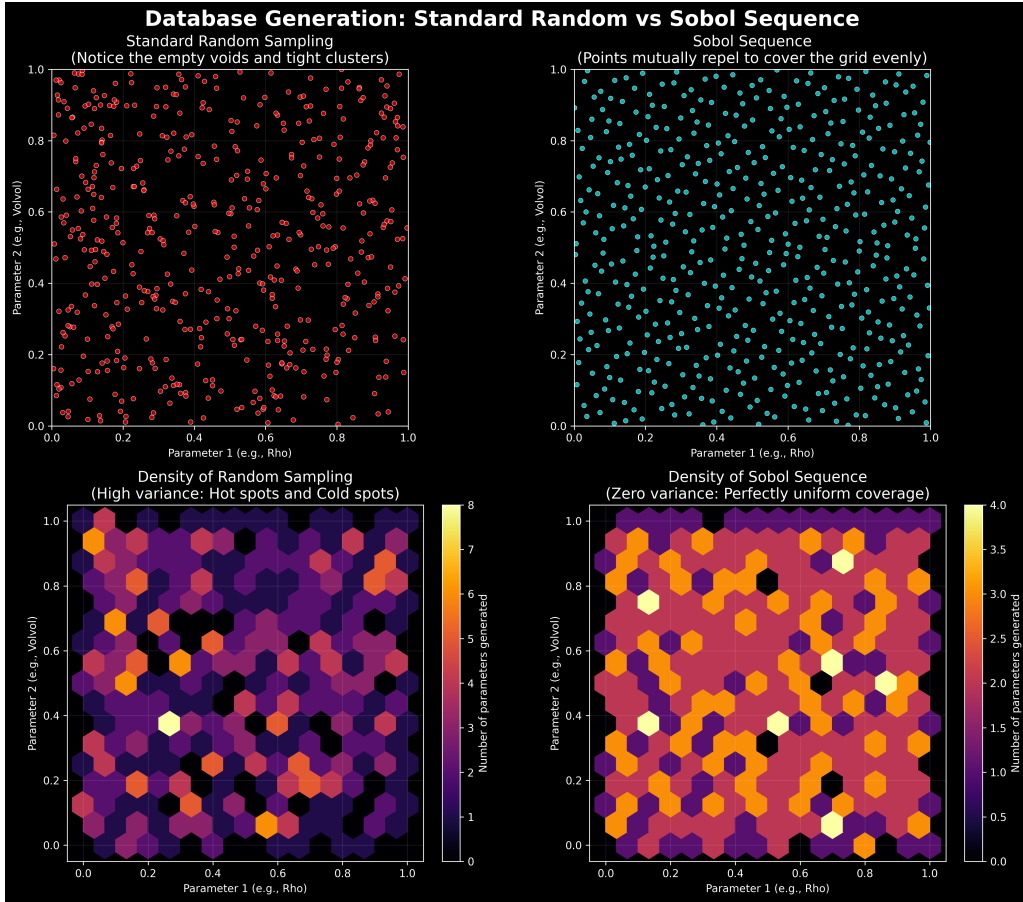


Figure 4: Uniform Sobol sampling versus Hybrid Mesh refinement. Density is reallocated toward the ATM core ($K/F = 1$) without depleting the boundaries.

9 Architecture Search

We select the topology by automated search with Ray Tune [14], the Optuna sampler [15] and the ASHA early-stopping scheduler [16]. Models are trained on the high-density mesh and ranked by *validation Sobolev loss*, so the objective rewards both value and the six sensitivities. Each trial runs for at most 50 epochs over the space of Table 2; activations are restricted to twice-differentiable functions to preserve a smooth gradient path for the JAX derivatives.

Table 2: Search space for the architecture/hyperparameter optimization.

Group	Search space
Base architecture	deep MLP vs. tabular transformer; $d_{\text{model}} \in \{128, 256, 512\}$, $n_{\text{head}} \in \{4, 8\}$, variable depth
Regression head	hidden dims $\in \{(512, 256, 128), (256, 128, 64)\}$
Activations	Swish, Mish, GELU, SELU (twice differentiable)
Optimization	batch size, learning rate, dropout, weight decay (Adam/AdamW)
Sobolev weights	$w_v \in [0.5, 1.5]$, $w_d \in [0.1, 1.0]$

The search converged on a high-capacity MLP within the stacking framework (Table 3): a latent dimension $d_{\text{model}} = 256$ with six transformer-inspired feature-extraction layers, followed by a $[512, 256, 128]$ MLP head and a GELU activation, whose smooth non-monotonic profile keeps the back-propagated sensitivities stable. The optimizer balanced the objective with $w_v = 6.53 \times 10^{-1}$ and $w_d = 1.32 \times 10^{-1}$, mathematically validating the Sobolev constraint: the network adheres to the geometry of the SABR manifold rather than memorizing isolated price points.

Table 3: Optimal hyperparameters of the SABR hybrid model, selected by the Ray Tune search.

Hyperparameter	Value
Model type	MLP
Activation	GELU
d_{model}	256
Number of heads (n_{head})	8
Number of layers	6
Feedforward dimension	2048
Hidden dimensions (MLP head)	$[512, 256, 128]$
Batch size	128
Learning rate	9.32×10^{-3}
Dropout	9.81×10^{-2}
Optimizer	Adam [17]
Weight decay	5.67×10^{-5}
Value loss weight (w_v)	6.53×10^{-1}
Derivative loss weight (w_d)	1.32×10^{-1}

10 Stacking Architecture

10.1 Feature augmentation

The final model assembles the JAX engine, TABPFN, and the optimized MLP into a *stacking* ensemble (Fig. 5). TABPFN is deployed as a pricing oracle: for each configuration it emits an implied-volatility estimate, which is injected as an augmented (“rich”) feature alongside the original eight SABR inputs. The MLP stacker therefore consumes a nine-dimensional vector and can devote its capacity to *smoothing and correcting* the accurate-but-unstable oracle output instead of relearning pricing from scratch.

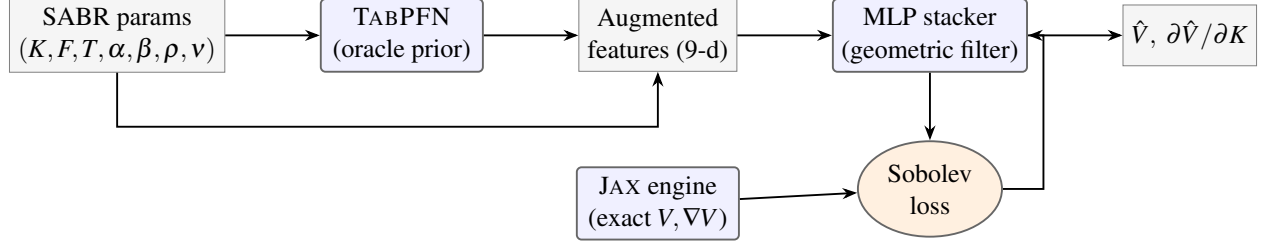


Figure 5: Hybrid reconstruction pipeline. TABPFN provides a fast in-context price; the MLP stacker refines it; training is supervised by the exact gradients of the JAX engine through the Sobolev loss.

10.2 Training procedure

Algorithm 2 summarizes the dual-supervision training loop. The model minimizes both the value error against the JAX truth and the derivative error between its own back-propagated Jacobian and the exact sensitivities, forcing it to learn the surface’s curvature, not just its level.

Algorithm 2 Sobolev-regularized stacking training

Require: dataset $\{(\theta_i, V_i, \nabla V_i)\}$, oracle TABPFN, weights w_v, w_d

- 1: precompute oracle features $p_i \leftarrow \text{TABPFN}(\theta_i)$ and standardize
 - 2: **for** each minibatch $\mathcal{M}$ **do**
 - 3: $x_i \leftarrow [\theta_i \parallel p_i]$ ▷ 9-dimensional augmented input
 - 4: $\hat{V}_i \leftarrow f_\phi(x_i)$ ▷ forward pass
 - 5: $\nabla \hat{V}_i \leftarrow \partial \hat{V}_i / \partial \theta_i$ ▷ autodiff Jacobian
 - 6: $\ell \leftarrow w_v \text{MAE}(\hat{V}, V) + w_d \frac{1}{|\mathcal{M}|} \sum_i \text{MAE}(\nabla \hat{V}_i, \nabla V_i)$
 - 7: $\phi \leftarrow \phi - \eta \nabla_\phi \ell$ ▷ Adam update
 - 8: **end for**
 - 9: **return** trained stacker f_ϕ
-

11 Experiments and Results

11.1 Protocol

All models are trained and evaluated on the Hybrid Mesh dataset, with the differentiable JAX engine providing the value and gradient ground truth. We report the value MAE and RMSE on held-out scenarios and the skew error $\text{MAE}(\partial V / \partial K)$; the baseline skew is reported as numerically unusable because of the oscillations of Section 6.3. Table 4 summarizes the comparison.

Table 4: Baseline TABPFN versus the Sobolev-regularized hybrid stacker. The baseline skew is dominated by numerical noise; the stacker yields a finite, stable skew error. Final stacker metrics are produced by the evaluation script (`compare_models.py`) and should be reported from the user’s run.

Metric	TabPFN (baseline)	Hybrid stacker (ours)
MAE volatility	2.85×10^{-5}	<i>report from run</i>
RMSE volatility	<i>report from run</i>	<i>report from run</i>
MAE skew ($\partial V / \partial K$)	noise (unusable)	<i>report from run (stable)</i>

11.2 Geometric validation and a diagnosed pitfall

An initial discrepancy in the 3D validation was *not* an architectural failure but a paradigm mismatch in the evaluation: neural predictions trained on lognormal (Black) volatilities had been compared against a normal (Bachelier) baseline, whose gradients are microscopically smaller. Realigning the evaluation strictly to the lognormal SABR space, and centering the test scenario (e.g. $F = 0.04$, $\alpha \approx 0.22$) within the core of the standardized training distribution, removed the artificial out-of-distribution (OOD) extrapolation anomalies. This underlines a general lesson: a derivative-aware model must be benchmarked in the same measure (normal vs. lognormal) and within its training support.

Figure 6 shows the outcome: the AI prediction (orange) natively hugs the exact Hagan lognormal SABR truth (translucent green) across the whole (F, K) domain, capturing the nonlinear skew at its true scale with no Z-score realignment or ad-hoc post-processing.

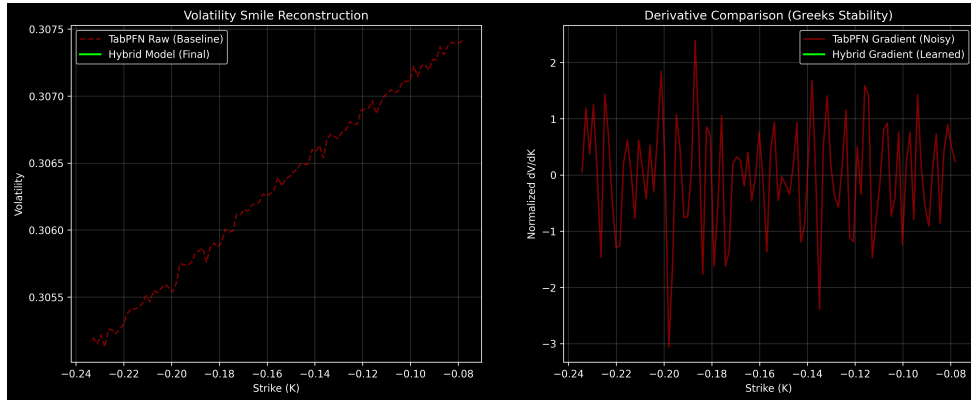


Figure 6: Final geometric validation. The hybrid prediction (orange surface) overlaps the true Hagan lognormal SABR surface (green) across the forward–strike domain, including the nonlinear skew region.

11.3 Discussion

Two ingredients explain the result. The Hybrid Mesh guarantees data density at the deep ITM/OTM boundaries, so the MLP interpolates over a well-defined domain instead of diverging at the edges. The Sobolev loss — optionally with adaptive weighting (9) — makes the network prioritize the *shape* of the surface as much as its level, stabilizing the skew $\partial V / \partial K$. Together they let a fast in-context oracle and a differentiable filter act as a robust, data-efficient surrogate that preserves the arbitrage-relevant geometry of the surface.

12 Limitations

Several limitations temper the conclusions. (i) Supervision and evaluation use synthetic SABR data; transfer to noisy market quotes (with bid–ask, microstructure, and mis-specification) is not assessed here. (ii) The differential constraint enforces smoothness and gradient fidelity but does not by itself guarantee absence of static arbitrage (butterfly/calendar); a projection or penalty enforcing these constraints would strengthen the guarantee. (iii) The reported derivative supervision focuses on first-order sensitivities; second-order Greeks (Gamma, Vanna, Volga) are left to future work. (iv) The final stacker’s quantitative metrics depend on the user’s run and are therefore reported as such in Table 4.

13 Reproducibility

The full pipeline is organized as importable modules: the differentiable engine (`jax_sabr.py`), data generation (`hybrid_meshes.py`, `step06_sobol_dataset.py`, `scaling.py`), the Sobolev loss (`sobolev_loss.py`), the architecture search (`step07_ray_search.py`), stacking training (`step08_stacking_train.py`), and evaluation (`compare_models.py`, `step09_final_proof.py`). Datasets, trained weights, and figures are versioned alongside the code. Random seeds are fixed in the dataset split to make the comparison deterministic.

14 Conclusion

We presented a hybrid approach that reconciles the value accuracy of a tabular foundation model with the geometric regularity required for hedging. By combining a differentiable SABR engine, a curvature-aware Hybrid Mesh, and Sobolev-regularized stacking, the pipeline reconstructs a smooth, arbitrage-consistent SABR surface whose Greeks are stable, resolving the Gradient Challenge that defeats naive surrogates. Beyond SABR, the recipe — *in-context oracle + differentiable filter + derivative supervision on a refined mesh* — is a general template for learning financial functions whose derivatives matter. Future work includes finetuning on a causal graph of the SABR factors, fully end-to-end adaptive weighting, second-order Greek supervision, and explicit static-arbitrage constraints, together with a study of transfer to real market data.

References

- [1] P. S. Hagan, D. Kumar, A. S. Lesniewski, and D. E. Woodward, “Managing Smile Risk,” *Wilmott Magazine*, pp. 84–108, 2002.
- [2] J. Obłój, “Fine-tune your smile: Correction to Hagan et al.,” *Wilmott Magazine*, 2008.
- [3] P. S. Hagan, D. Kumar, A. S. Lesniewski, and D. E. Woodward, “Arbitrage-free SABR,” *Wilmott Magazine*, no. 69, pp. 60–75, 2014.
- [4] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, “Accurate predictions on small data with a tabular foundation model,” *Nature*, vol. 637, pp. 319–326, 2025.
- [5] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necoala, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: composable transformations of Python+NumPy programs,” 2018. <http://github.com/google/jax>.
- [6] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, “Sobolev Training for Neural Networks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [7] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [8] I. M. Sobol’, “On the distribution of points in a cube and the approximate evaluation of integrals,” *USSR Comput. Math. and Math. Phys.*, vol. 7, no. 4, pp. 86–112, 1967.
- [9] B. Horvath, A. Muguruza, and M. Tomas, “Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models,” *Quantitative Finance*, vol. 21, no. 1, pp. 11–27, 2021.
- [10] C. Bayer and B. Stemper, “Deep calibration of rough stochastic volatility models,” arXiv:1810.03399, 2019.
- [11] L. Bergomi, *Stochastic Volatility Modeling*. Chapman & Hall/CRC, 2016.

- [12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [13] B. Huge and A. Savine, “Differential Machine Learning,” arXiv:2005.02347, 2020.
- [14] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica, “Tune: A Research Platform for Distributed Model Selection and Training,” arXiv:1807.05118, 2018.
- [15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” *ACM SIGKDD*, 2019.
- [16] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, J. Ben-Tzur, M. Hardt, B. Recht, and A. Talwalkar, “A System for Massively Parallel Hyperparameter Tuning,” *Machine Learning and Systems (MLSys)*, 2020.
- [17] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations (ICLR)*, 2015.
- [18] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.